Telomy — Device Interception & Jailbreak Reference

Prepared 29 Oct 2026. Deep sweep of open-source repos, protocol teardowns, and tooling for pulling raw/near-raw biometrics (HR, HRV/RR-IBI, raw PPG, ECG, SpO_2, skin-temp, accel, sleep, glucose) off consumer wearables and CGMs into Telomy's stack: iOS CoreBluetooth app, Android companion (adb), FastAPI backend, and the existing Nightscout ingest.

> **Accuracy note:** star counts and "last-commit" dates are approximate (GitHub doesn't render exact dates); verify on the repo page before quoting externally. Technical capability/method claims are high-confidence and sourced. Everything here assumes **device-owner consent** (our model); BLE interop leans on DMCA §1201(f).

0. The three things that matter most

- **Raw PPG is almost never exposed over BLE.** Every mainstream device (WHOOP API, Ultrahuman, RingConn, Pebble SDK, Apple, Samsung-without-partner) gives *processed* HR/SpO_2/HRV scalars. The **only cheap device that streams a genuine raw optical waveform is the Colmi R02** (after a community firmware mod). For validated raw ECG+RR, the **Polar H10** is the gold standard.
- **No sub-$300 wrist/ring gives true beat-to-beat RR/IBI over BLE** — except anything exposing the **standard BLE Heart Rate Service (0x180D / char 0x2A37)**, whose RR-interval field is the universal, license-free, CoreBluetooth-native path. The Polar H10 does both (0x180D RR *and* raw ECG via its PMD service).
- **Licensing is the real trap.** The richest code — **Gadgetbridge (AGPLv3)** and **OpenStrap (AGPL-3.0)** — is network-copyleft. **Reference only, clean-room reimplement, never vendor into Telomy.** The MIT-licensed projects (Colmi client, whoop-ble-swift, pydexcom, nightscout-librelink-up) are safe to build on with attribution.

Decision table

DEVICE	HACKABLE?	BEST REPO / METHOD	LICENSE	RAW DATA YOU ACTUALLY GET	STACK FIT
Polar H10 / Verity	✅✅ Fully open	`polarofficial/polar-ble-sdk` + PMD	SDK permissive	**Raw ECG 130 Hz, ACC, RR/PPI**	iOS + Android (native SDK)
Any 0x180D strap	✅✅ Standard	roll-your-own 0x2A37 parser	n/a	HR + **RR intervals (IBI)**	iOS CoreBluetooth native
Colmi R02/R03/R06	✅✅ No-auth GATT	`tahnok/colmi_r02_client` + Edge Impulse mod	MIT	HR, SpO_2, sleep; **raw PPG + accel after firmware flash**	iOS native (like our TK3040)
WHOOP 4.0	✅ RE'd	`Asherlc/whoop-ble-swift` (MIT) / OpenStrap (ref)	MIT / AGPL	HR, **RR, SpO_2, temp, accel**; raw PPG via OpenStrap	iOS + Android (our NOOP)
WHOOP 5.0 / MG	◐ RE'd, bonded	OpenStrap `protocol` (reference)	AGPL	raw PPG/RR/ECG **once OS-bonded**; else 0x180D HR only	needs bond work
Galaxy Watch 8	◐ On-watch app	Wear OS Health Services + Sensor SDK	Apache / Samsung EULA	HR+HRV live; **raw PPG/IBI only via partner SDK**	Android companion only
Ultrahuman Ring Air	◐ RE'd (processed)	Gadgetbridge protocol / Partner API	AGPL ref	HR, HRV *scalar*, SpO_2, skin-temp (5-min)	iOS poll; no raw PPG
RingConn Gen2/3	◐ Auth cracked	`perezjuanj/OpenCircuit` (Swift→HealthKit)	check repo	HR, HRV, SpO_2, resp, temp (processed)	**best Swift reference design**
Pebble (Core Time 2)	◐ SDK/firmware	`coredevices/PebbleOS` (Apache)	Apache-2.0	BPM via SDK; **raw PPG only if you fork firmware**	companion over BLE
Apple Watch S11	✗ for raw	HealthKit (HKWorkoutSession, HKHeartbeatSeries, ECG)	—	HR, IBI (HeartbeatSeries), ECG voltage; **no raw PPG**	iOS native
Garmin	✗ live raw	`GarminDB` (cloud) / Connect IQ	GPL / SDK	HR/HRV/SpO_2/sleep (synced, not live raw)	backend/cloud
FreeStyle Libre 2 Plus	✅ (India: see caveat)	LibreLinkUp or dongle+xDrip / Juggluco	MIT / GPL	glucose ~1–5 min → Nightscout → Telomy	Android capture
Dexcom G6/G7	✅ but not in India	`xDrip+` native BLE / `pydexcom`	GPL / MIT	glucose 5 min	not India-viable
Noise Luna / boAt ring	✗	— (closed, cloud-locked)	—	nothing	skip

Buy list (updated): Polar H10 (HRV/ECG reference) + **Colmi R02 ×2–3** (cheapest raw-PPG ring, ~₹1.5–2.5k) + **Galaxy Watch 8** (wrist, if wanted) + **FreeStyle Libre 2 Plus** (CGM, via the India-safe path below). Apple Watch as a HealthKit convenience feed only.

1. Wrist & chest wearables

Polar H10 / Verity Sense — the best raw target, start here

- `polarofficial/polar-ble-sdk` — ~1.5k★, active 2026, permissive SDK license. Official Swift (iOS) **+** Android SDK wrapping the **PMD (Polar Measurement Data)** GATT service. **H10:** ECG 130 Hz, ACC, HR, RR/PPI. **Verity:** PPG, PPI, ACC. In-repo `technical_documentation/` has the PMD byte layout if you want to skip the SDK and talk PMD directly in CoreBluetooth (service `FB005C80-02E7-F387-1CAD-8ACD2D8DF0C8`, control `…81`, data `…82`).
- Python prototyping: `fsmeraldi/bleakheart` (MPL-2.0, safe as a dependency) does generic 0x180D HR+RR *and* Polar ECG/ACC/PPG; `kbre93/OpenHRV` (MIT) for H10 RR biofeedback.
- **Verdict:** highest ROI in the whole report. Adopt the SDK in both apps; it's the apples-to-apples comparator for the KYECG chest patch.

Standard BLE Heart Rate Service (0x180D / 0x2A37) — the universal path

Maps 1:1 onto CoreBluetooth. **Reusable RR-interval decode (keep this):**

> Char `0x2A37` notify payload: **byte 0 = flags**. bit0 → HR is uint8 (0) or uint16-LE (1); bit3 → energy-expended (uint16-LE) present; **bit4 → RR-intervals present**. After HR (and optional energy), the remaining bytes are one-or-more **uint16-LE RR values in units of 1/1024 s** → `rr_ms = raw × 1000 / 1024`. Multiple RR per notification is normal. Identical logic in Swift `Data`, Kotlin `ByteArray`, JS `DataView(...,true)`, Python `struct '<H'`.

References: Adafruit `Adafruit_CircuitPython_BLE_Heart_Rate`, Chrome Web-Bluetooth samples, `fsmeraldi/bleakheart` (Python), `chbrown/BluetoothLE-HeartRate` (Node, MIT, dormant).

Samsung Galaxy Watch 8 / Wear OS

- **Hard truth:** no raw PPG over BLE; everything runs *on-watch*. Raw PPG/IBI is gated behind the **Samsung Health Sensor SDK** partner program (approval needed only to *distribute* — **developer mode on your own watch unlocks raw PPG/IBI for an internal rig**).
- Supported, no-approval path: **Wear OS Health Services** (`androidx.health.services`) + `android/health-samples` (Apache-2.0) → on-watch app streams HR + available HRV to the phone → FastAPI. **Health Connect** reads HR/HRV(RMSSD)/SpO_2/skin-temp after sync (Samsung's HRV→Health Connect sync is unreliable — don't depend on RMSSD arriving there).
- **Blueprint repo:** `Kaist-ICLab/EnPULSE` (Apache-2.0, actively maintained) — a full watch+phone+backend platform pulling **raw PPG/IBI/accel/ECG/EDA/skin-temp @ 25 Hz** via the Sensor SDK, and it documents the **Developer-Mode-in-Samsung-Health unlock that avoids the partner license for your own devices** (the `Error 2003` gotcha). `Kaist-ICLab/GalaxyPPG-Logger` + `-Supplementary-Code` are the focused 25 Hz PPG logging + validated PPG→HRV processing references.
- **Android-companion workstream only — no CoreBluetooth route to the watch itself.** The *only* iOS path is to build a **custom GATT peripheral into the Wear OS watch app** and have our iOS app be the central; otherwise iOS consumes this data via the backend.

Apple Watch Series 11

- **Raw PPG = jailbreak only = not viable.** HealthKit is the ceiling: **HKWorkoutSession keepalive** for live HR (~1–5 s), **HKHeartbeatSeriesQuery** for beat-to-beat IBI, **HKElectrocardiogramQuery** for ~512 Hz 30-s lead-I ECG voltage. **Health Auto Export** (commercial) is the ship-tomorrow HealthKit→REST/JSON stopgap. Pure iOS workstream; secondary convenience feed, not the comparator.

Garmin

- No live raw BLE (proprietary GFDI). Realistic: `tcgoetz/GarminDB` (GPL, cloud-sync → SQLite; TOS-dicey), **Connect IQ** on-watch app for live HR, `python-fitparse` / `fitdecode` for FIT files. Manage expectations — cloud/FIT, not live raw. Gadgetbridge has growing Garmin support but is AGPL (reference only).

---

2. WHOOP — 4.0 reachable, 5.0 now mapped (behind a bond)

- **OpenStrap** (`OpenStrap/edge` app + `OpenStrap/protocol` spec + `OpenStrap/analytics`) — **AGPL-3.0, reference only.** The flagship and the first public crack of the WHOOP **5.0/MG "fd4b"** generation: documents **raw green/red/IR PPG, accelerometer, RR intervals, SpO_2, skin-temp, and MG ECG records**, plus the token-ack history-offload handshake. Covers 4.0 + 5.0 + MG. Flutter (iOS TestFlight + Android).
- MIT building blocks (safe to use): `Asherlc/whoop-ble-swift` (native Swift/CoreBluetooth, `AsyncStream` of HR/RR/IMU — your cleanest iOS start; requires an OS-bonded strap), `christianmeurer/whoop-reader` (Python, decodes the 96-byte realtime packet: HR/RR/SpO_2/temp), `app-noop/noop` (desktop, 4.0 tested / 5.0 experimental — *unrelated* to our NOOP).
- 4.0 custom service chars: `61080002` write / `…03` responses / `…04` events / `…05` data stream; plaintext, CRC-32, no hard bond. **The 5.0 gate is an OS-level encrypted bond** — pre-bond you only get standard **0x180D live HR (no RR)**; near-raw needs the bond (still being RE'd).
- Official **WHOOP Developer API v2** (OAuth2) = aggregates only (HRV as one RMSSD value, RHR, SpO_2, sleep totals) — `felixnext/whoopy` (MIT) is a clean Python client. **No raw data.** Gadgetbridge does **not** support WHOOP.
- **Verdict (revises the earlier "don't rebuy"):** WHOOP 5.0 raw is now feasible via OpenStrap's field map — reference it, clean-room the decoders into our NOOP (Kotlin) + iOS (Swift, seeded by whoop-ble-swift), and invest in the 5.0 bond. Keep the existing 4.0 rig; a 5.0 unit is now worth it *if* we fund the bond work.

---

3. Pebble (Core Time 2 / Core 2 Duo + Rebble)

- **Not an interception target — an SDK/firmware target.** `coredevices/PebbleOS` (Apache-2.0, ~1.5k★, very active) is the open firmware you can legally fork. Stock **HealthService API gives BPM only**; **raw PPG requires a firmware patch** (feasible precisely because it's open — a genuine differentiator). Core Time 2 has optical HR; Core 2 Duo does not. Companion via PebbleKit BLE. Pursue only if a wrist device is on the roadmap and you want a legally-forkable raw-PPG path.

---

4. Smart rings (India focus)

Colmi R02 / R03 / R06 / R10 — RECOMMENDED cheap test unit

- **~₹1,500–2,500 on Amazon.in/Flipkart** (verify live INR — Amazon.in blocked our fetch; global floor ~$16.50). Companion app **QRing** (by Yawell) — *if the listing says QRing, it's the hackable variant.*
- `tahnok/colmi_r02_client` (MIT, ~667★, active) — **no pairing/security keys.** Nordic-UART-style service `6E40FFF0-…`, write `…0002`, notify `…0003`, 16-byte packets (byte0=command, byte15=checksum mod 255) — nearly identical framing to our **TK3040 0x18** work, trivial to port to Swift CoreBluetooth. Stock exposes real-time HR, SpO_2, logs, steps, sleep. **No HRV/IBI/raw PPG on stock.**
- `puxtril/colmi-docs` (hosted at colmi.puxtril.com) — the **command-byte Rosetta Stone for the whole 0x18/QRing family**, reversed from QRing on an R03: `0x01` set-time, `0x03` battery, `0x15` HR-log, `0x1E` realtime HR, `0x37` stress, `0x39` **HRV**, `0x43` steps, `0x44` sleep, and the streaming-subscription mechanism `0x69` **Data-Request /** `0x6A` **Stop** (DataType 3=SpO_2, 6=realtime HR; DataAction start/pause/continue/stop). This is the closest analog to our TK3040 0x18 opcodes — read it alongside the client before porting. A "Big Data" bulk/near-raw history page is worth a human read.
- `atc1441/ATC_RF03_Ring` (~494★) — chip teardown: the rings run a **BlueX RF03 SoC with unsigned, OTA-downloadable firmware** (SWD pads solderable) — *why* the whole family is hackable and how custom firmware exposes raw sensor rates the stock protocol hides.
- **Raw path (the differentiator):** `edgeimpulse/example-data-collection-colmi-r02` ships `ring.py` logging **`ppg_raw`, accX/Y/Z, spO2_raw** to CSV after flashing `R02_3.00.06_FasterRawValuesMOD.bin` via the ATC_RF03 web flasher. A **time-varying optical waveform you run your own peak-detector/IBI/HRV on.** Forks: `qlam-ai/qring`, `patmorli/colmi-r09`.
- Gadgetbridge rates R02/R06 "highly supported" (R03 = same HW). **Warning:** some R02/R03/R06 units ship different hardware + a different app and are NOT supported — confirm QRing before buying; buy 2–3 (sizing + variant risk).

Ultrahuman Ring Air (Indian company)

- ~₹19,999 (MRP ₹28,499), widely available. BLE **reverse-engineered** (Gadgetbridge protocol doc): opcode 0x04 returns 32-byte records with HR, **HRV scalar**, SpO_2, skin-temp (float °C), steps, stress ~every 5 min. **Processed only — no raw PPG/IBI/accel.** Official **OAuth2 Partner API** (gated) + a separate **UltraSignal dev kit** that reportedly streams raw PPG/temp/accel via a loaned unit — worth a direct partnership inquiry (they're Indian, reachable).

RingConn Gen 2/3 — best Swift reference, not India-stocked

- Import only ($299+). `perezjuanj/OpenCircuit` fully cracked the per-connection **SM3 challenge auth (keyed on the ring's MAC)** — connects with no account/app. It's a **Swift + CoreBluetooth → HealthKit** app decoding on-device: HR, **HRV**, SpO_2, resp rate, skin-temp, sleep (processed; no raw PPG/IBI). **Study its architecture as our reference design.** ⚠ **License is PolyForm Noncommercial 1.0.0 (not MIT) — commercial reuse needs a separate license; read the SM3-auth writeup for method, don't vendor the code.**

Oura Ring Gen 3/4/5 — raw-PPG reference (not for shipping)

- `Th0rgal/open_oura` (MIT, ~659★) — the **richest raw ring stream documented anywhere**: raw PPG, IBI, temperature, motion, SpO_2 samples. Method is heavy (BLE sniff + APK decompile + Ghidra port of `libringeventparser.so`, with a **per-connection 16-byte auth key**). Not shippable (proprietary IP, per-device auth), but the **raw-PPG packet framing and Ghidra methodology are gold for our own decoder work** and show what "good" raw ring data looks like. Oura's scores are computed on-phone (proprietary PyTorch), not on the ring.

Skip: Noise Luna Ring (~₹15–19k), boAt Smart Ring (₹8,999)

- Closed, cloud-locked, no public API or BLE RE. 91mobiles confirms "no way to interact with the ring" outside the app. Not worth it vs a Colmi. Generic Amazon.in rings that use the **QRing** app are Colmi-compatible rebrands (cheap spares); avoid ones naming a bespoke app. `jphein/smartring` handles Lefun-protocol rings.

---

5. CGMs → Nightscout → Telomy

Confirmed chain: Telomy already exposes a Nightscout-compatible endpoint — `GET/POST /nightscout/api/v1/entries` (main.py ~861–878, parsed by `cgm.py`, auth = SHA-1 of `TELOMY_NIGHTSCOUT_SECRET` in the `api-secret` header, or `?token=`). Any uploader targeting base URL `https://<server>/nightscout` works. Secret must be **≥12 chars**.

⚠ India correction (changes yesterday's rec)

Direct phone-to-sensor BLE for Libre 2 (**Juggluco / xDrip+ native**) was only reverse-engineered for the **EU-market** sensor. **India sensors are non-EU firmware**, so direct-BLE is a *gamble*. India-robust paths:

- **LibreLinkUp cloud-follower — recommended first for India.** `timoschlueter/nightscout-librelink-up` (MIT, ~293★, active, Docker). `LINK_UP_REGION=AP` works regardless of sensor firmware; mirrors Abbott's cloud → Nightscout. ~5-min poll + cloud lag; depends on the official app +

Abbott servers.

- **NFC→BLE transmitter dongle (MiaoMiao/Bubble, ~$30–60) + xDrip+** — fully local, region-independent (dongle re-reads over NFC), 5-min values.
- **Juggluco / xDrip+ direct BLE** — try (Juggluco first, NFC activate), keep 1 or 2 as fallback.

Sensors & tools

- **FreeStyle Libre 2 Plus** — sold in India, **~₹4,600–5,100 / 14-day sensor** (Apollo ₹4,624; Amazon.in ~₹5,100). This is the India default. `Juggluco` (GPL, ~476★) and `xDrip+` (GPL, ~1.8k★) do native BLE (EU-firmware caveat above); both upload to Nightscout natively.
- **Libre 3 / 3 Plus** — **not officially sold in India** (Abbott India lists only Libre 2 Plus + 14-day Libre). Only `Juggluco` pushes it to Nightscout natively; the `maheini/FreeStyle-Libre-3-patch` repo is **archived (22 Feb 2026)**; iOS = LibreLoop/LibreCRKit (feeds Loop, not Nightscout). Noisier, more firmware-churn. **Favor Libre 2 Plus.**
- **Dexcom G6/G7** — **effectively not sold in India** (no en-IN store, grey-market only). `xDrip+` native BLE (G6 mature; G7/ONE+ fragile) or `pydexcom` /Nightscout bridge (Share cloud, lag). Deprioritize for India; BYOD-only.
- Other India e-commerce CGMs: **Sibionics GS1** (interception immature), **Sugarfit/BeatO** programs often ride on a **Libre 2** sensor → the Juggluco/LibreLinkUp path still works regardless of the branded app.

Exact config — Libre 2 → Juggluco → Nightscout → Telomy

- Install Juggluco APK; NFC-scan the sensor to start it; let BLE connect.
- Menu → Settings → Exchange data → **Uploader**: enable **v1**, disable "Test v3" and "Send amount" (Telomy implements the v1 API).
- **Nightscout URL:** `https://<server>/nightscout` (no trailing slash).
- **Password/secret:** plain `TELOMY_NIGHTSCOUT_SECRET` (Juggluco SHA-1-hashes it to `api-secret`).
- Entries POST to `/nightscout/api/v1/entries` at ~1-min cadence → `cgm.py` ingests → `signals.glucose_mean` (needs ≥48 readings/day before a daily mean is emitted). Dedup on timestamp, so backfill is safe.

(xDrip+ equivalent: Settings → Cloud Upload → Nightscout Sync REST-API → Base URL `https://<SECRET>@<server>/nightscout/api/v1/`; supports multi-URL so a user can feed Telomy *and* a personal Nightscout.)

---

6. BLE reverse-engineering toolchain (to crack any new device)

- **nRF Connect** (freeware) — enumerate services/chars; `6E40…` → Colmi/TK class, `0xFE…` → Da-Fit class, `0x6108…` → WHOOP-like.
- **Android Bluetooth HCI snoop log** (built-in, fits our adb workflow) — enable in Developer Options, drive the vendor app, `adb pull btsnoop_hci.log` → **Wireshark** (filter `btatt.handle==0xNN`). Primary capture method, no extra hardware. (Android 16 blocked live capture; **Android 17 restores it.**)
- **Frida** (+ `objection`) — hook `setValue` / `onCharacteristicChanged` in the live vendor app to read plaintext values + encryption keys before they hit the air; `jadx` decompiles the APK for UUIDs/command tags/CRC logic; `apktool` repackages for non-rooted Frida-gadget injection.
- **Over-the-air sniffers** (only for bonded/encrypted links — rare on cheap rings): `nccgroup/Sniffle` (GPL, ~1.2k★, SONOFF CC2652P dongle ~$10) or **Nordic nRF Sniffer** (nRF52840 dongle ~$10).

Workflow: explore → HCI-snoop capture → Wireshark decode → Frida/jadx if obfuscated → sniffer only if bonded → reimplement natively (CoreBluetooth if unauthenticated GATT, else Android-companion→FastAPI push like NOOP).

---

7. Licensing — carry these forward

- **AGPL-3.0 (reference only, never copy into Telomy):** Gadgetbridge, OpenStrap (edge/protocol/analytics), Ultrahuman protocol doc is in Gadgetbridge. Reading for protocol knowledge is fine; clean-room reimplement.
- **GPL (keep at arm's length):** Juggluco, xDrip+, GarminDB, Sniffle.
- **MIT / permissive (safe to build on, attribute):** `tahnok/colmi_r02_client`, `qlam-ai/qring`, `Asherlc/whoop-ble-swift`, `christianmeurer/whoop-reader`, `app-noop/noop`, `felixnext/whoopy`, `pydexcom`, `nightscout-librelink-up`, `Th0rgal/open_oura`, `zHElEARN/polar-python`, `cyberjunky/python-garminconnect`, `coredevices/PebbleOS` (Apache), `android/health-samples` + `Kaist-ICLab/EnPULSE` (Apache), `edgeimpulse/...colmi-r02` (BSD-3-Clear). `fsmeraldi/bleakheart` = MPL-2.0 (file-level copyleft, OK as dependency). `polar-ble-sdk` = Polar's own NOASSERTION license (permits commercial use with notice — legal to review).
- **PolyForm Noncommercial (NOT free for a commercial product):** `perezjuanj/OpenCircuit` (RingConn) — method/reference only unless separately licensed.
- **No license stated (read-only reference, don't copy):** `bWanShiTong/reverse-engineering-whoop-post`.
- **ToS/warranty:** all of this is off-label; WHOOP private-cloud-API scraping is the clearest ToS violation + credential liability — avoid. Native BLE/on-device decode is the defensible path.

---

8. Recommended build order

- **Polar H10 via `polar-ble-sdk`** (iOS+Android) — raw ECG+RR now, zero legal risk. The HRV/ECG reference lane.
- **Generic 0x180D/0x2A37 parser** in CoreBluetooth (+ Bleak mirror) — one parser, HR+RR from a whole class of straps.
- **Colmi R02** — port `colmi_r02_client` to CoreBluetooth (mirrors our TK3040 0x18); flash the Edge Impulse mod for raw PPG and derive our own IBI/HRV. Cheapest raw-waveform ring.

- **CGM: Libre 2 Plus → LibreLinkUp (AP region) → Nightscout → Telomy** for India; dongle+xDrip for fully-local; direct Juggluco as a try-first.
- **Galaxy Watch 8** on-watch Health Services app (fork `android/health-samples`); apply to the Samsung Sensor SDK partner program for raw PPG/IBI.
- **WHOOP 5.0** — reference OpenStrap's fd4b field map, clean-room into NOOP + iOS, fund the bond crack. Keep the 4.0 rig meanwhile.
- **Apple Watch** HealthKit (HKWorkoutSession + HeartbeatSeries + ECG) / Health Auto Export as a convenience feed.
- **RingConn OpenCircuit** — study as the Swift/CoreBluetooth→HealthKit reference architecture.